

Práctica 2: Búsquedas con Costes y Traza

1. Introducción

El objetivo de esta práctica es implementar distintos algoritmos de búsqueda para resolver laberintos. Partiendo del código de la práctica anterior, del código proporcionado en el enunciado de la práctica y de los pseudocódigos de los distintos algoritmos de búsqueda explicados en clase, he implementado algoritmos que calculan el camino desde la entrada (E) hasta la salida (S).

Partimos de la base de conceptos aprendidos en clase como que los algoritmos que vamos a implementar trabajan con dos listas: abiertos y cerrados, y que cada uno inserta nodos en la lista de abiertos de forma distinta.

Los algoritmos que se implementan son:

- Búsqueda en anchura: explora nivel por nivel, usa FIFO
- Búsqueda en profundidad: explora el camino completo antes de retroceder, usa LIFO
- Búsqueda A*: usa una heurística para ir hacia la meta más rápido

2. Diseño de los Algoritmos

2.1 Estructura del código y temario de la asignatura

- Clase Búsqueda: base común de todos los algoritmos, porque cuando empecé a implementarlos me di cuenta que eran casi idénticos, salvo por como se insertan los sucesores en Abiertos. Lo implementé como en las diapositivas del tema 3: crear listas de Abiertos y Cerrados, meter el nodo inicial en Abiertos, mientras Abiertos no esté vacío saco el primero al Actual, meto Actual en Cerrados, si Actual es la meta devuelvo el camino y sino genero sucesores, quito repetidos e inserto en Abiertos, si Abiertos se vacía no hay solución.
- Clase Búsqueda en Anchura: nodos se insertan al final de Abiertos y se sacan por el principio, siempre encuentra una solución si existe, encuentra la solución de menor coste si todos los pasos cuestan lo mismo y como se indica en el tema 3 de esta asignatura, tiene complejidad temporal y espacial $O(r^{(p+1)})$.
- Clase Búsqueda en Profundidad: nodos se insertan al principio de Abiertos, según el tema 3, no garantiza encontrar la solución óptima porque si lo encuentra la meta en profundidad no explora opciones con menos coste, usa menos memoria que Anchura, puede no terminar si el grafo es infinito y no hay limite de profundidad

- Clase Búsqueda A*: búsqueda informada, usa la función $f(n) = g(n) + h(n)$, donde g es el coste real desde el inicio hasta el nodo actual y h la heurística (estimación del coste desde el nodo actual hasta la meta). Como heurística he usado la distancia Manhattan que es la suma de las diferencias absolutas de filas y columnas entre el nodo actual y la meta. Mantengo Abiertos ordenado por valor de menor a mayor con el método insertarOrdenado que recorre la lista hasta encontrar la posición correcta.
- Clase BusquedaA*Mejorado: igual que A* pero he añadido a la heurística los costes de moverse a cada dirección multiplicando las diferencias por 1 o 2 según corresponda. Esto hace que expanda menos nodos y que la estimación sea más ajustada, pero en MAPA_GRANDE no es admisible

2.2 Estructuras de Datos Usadas

He usado LinkedList para implementar la cola y la pila en lugar de usar Stack o Queue por separado porque encontré que esta estructura me permite añadir elementos al principio y al final.

- Búsqueda en anchura: FIFO, uso addLast para meter y removeFirst para sacar, así salen en el mismo orden que entraron.
- Búsqueda en profundidad: LIFO, uso addFirst para meter y removeFirst para sacar, así el último elemento en entrar es el primero en salir
- Búsqueda A*: mantengo la lista ordenada con “insertarOrdenado”, que sería como una cola de prioridad y el primero de la lista siempre es el nodo con menor valor.

Para los cerrados sigo usando LinkedList, añadiendo con addLast y consultando con contains.

Los métodos usados para esta estructura son: addLast, addFirst, removeFirst, size, contains, get, add, isEmpty

2.3 Cálculo de Costes

- Coste real: g , son los pasos que da desde el inicio al nodo actual, lo he calculado sumando el coste correspondiente al coste de cada movimiento en getSucesores, como se especificó en el enunciado (Arriba: 2, izquierda: 2, abajo: 1, derecha: 1). El cambio de coste uniforme 1 a estos costes hace que el anchura ya no garantice el camino óptimo porque asume coste uniforme. Solo el A* sigue encontrando el óptimo.

2.4 Capturas de la traza (pasos de anchura y profundidad con el MAPA_CALLEJON)

```
Iniciando busquedaAnchura
Origen: (1,1) -> Meta: (4,6)
[PASO 1]
Seleccionado: (1,1) [f=0, g=0, h=0]
Generados hijos: (2,1), (1,2)
Estado ABIERTOS: [(2,1), (1,2)]
Estado CERRADOS: [(1,1)]

[PASO 2]
Seleccionado: (2,1) [f=1, g=1, h=0]
[REPETIDO DESCARTADO]: (1,1)
Generados hijos: (3,1)
Estado ABIERTOS: [(1,2), (3,1)]
Estado CERRADOS: [(1,1), (2,1)]

[PASO 3]
Seleccionado: (1,2) [f=1, g=1, h=0]
[REPETIDO DESCARTADO]: (1,1)
Generados hijos: (1,3)
Estado ABIERTOS: [(3,1), (1,3)]
Estado CERRADOS: [(1,1), (2,1), (1,2)]

[PASO 4]
Seleccionado: (3,1) [f=2, g=2, h=0]
[REPETIDO DESCARTADO]: (2,1)
Generados hijos: (4,1)
Estado ABIERTOS: [(1,3), (4,1)]
Estado CERRADOS: [(1,1), (2,1), (1,2), (3,1)]

[PASO 5]
Seleccionado: (1,3) [f=2, g=2, h=0]
[REPETIDO DESCARTADO]: (1,2)
Generados hijos: (1,4)
Estado ABIERTOS: [(4,1), (1,4)]
Estado CERRADOS: [(1,1), (2,1), (1,2), (3,1), (1,3)]

[PASO 6]
Seleccionado: (4,1) [f=3, g=3, h=0]
[REPETIDO DESCARTADO]: (3,1)
Generados hijos: (4,2)
Estado ABIERTOS: [(1,4), (4,2)]
Estado CERRADOS: [(1,1), (2,1), (1,2), (3,1), (1,3), (4,1)]
```

```
Iniciando busquedaProfundidad
Origen: (1,1) -> Meta: (4,6)
[PASO 1]
Seleccionado: (1,1) [f=0, g=0, h=0]
Generados hijos: (2,1), (1,2)
Estado ABIERTOS: [(1,2), (2,1)]
Estado CERRADOS: [(1,1)]

[PASO 2]
Seleccionado: (1,2) [f=1, g=1, h=0]
[REPETIDO DESCARTADO]: (1,1)
Generados hijos: (1,3)
Estado ABIERTOS: [(1,3), (2,1)]
Estado CERRADOS: [(1,1), (1,2)]

[PASO 3]
Seleccionado: (1,3) [f=2, g=2, h=0]
[REPETIDO DESCARTADO]: (1,2)
Generados hijos: (1,4)
Estado ABIERTOS: [(1,4), (2,1)]
Estado CERRADOS: [(1,1), (1,2), (1,3)]

[PASO 4]
Seleccionado: (1,4) [f=3, g=3, h=0]
[REPETIDO DESCARTADO]: (1,3)
Generados hijos: (1,5)
Estado ABIERTOS: [(1,5), (2,1)]
Estado CERRADOS: [(1,1), (1,2), (1,3), (1,4)]

[PASO 5]
Seleccionado: (1,5) [f=4, g=4, h=0]
[REPETIDO DESCARTADO]: (1,4)
Generados hijos: (1,6)
Estado ABIERTOS: [(1,6), (2,1)]
Estado CERRADOS: [(1,1), (1,2), (1,3), (1,4), (1,5)]

[PASO 6]
Seleccionado: (1,6) [f=5, g=5, h=0]
[REPETIDO DESCARTADO]: (1,5)
Generados hijos: (1,7)
Estado ABIERTOS: [(1,7), (2,1)]
Estado CERRADOS: [(1,1), (1,2), (1,3), (1,4), (1,5), (1,6)]
```

3. Bugs encontrados durante la implementación

La mayor parte de mis fallos a lo largo del código fueron a la hora de hacer uso de las nuevas estructuras aprendidas para java (lista, cola, LinkedList...).

3.1 StringIndexOutOfBoundsException al iniciar

El mapa que tenía en Tron.java tenía filas de distinto ancho, así que cuando una fila era más corta se salía del índice

3.2 New List() no me compilaba

Al principio quise implementar las listas de Abiertos y Cerrados con new List(), pero me di cuenta que List es una interfaz en Java y no se puede instanciar directamente, así que decidí usar LinkedList que es una clase que implementa List.

Código con el error:

```
List abiertos = new List();
```

Solución:

```
LinkedList<Nodo> abiertos = new LinkedList<>();
```

3.3 addLast con dos parámetros

En un momento usé addLast pasándole los abiertos y sucesores, pero me di cuenta que addLast solo acepta un parámetro (el elemento que quiero añadir)

Código con el error:

```
abiertos.addLast(abiertos, sucesores.get(i));
```

Solución:

```
abiertos.addLast(sucesores.get(i));
```

3.4 Los tres algoritmos daban los mismos resultados

- Private no sobrescribe
private void insertarEnAbiertos(LinkedList<Nodo> abiertos, Nodo nuevo)
protected void insertarEnAbiertos(LinkedList<Nodo> abiertos, Nodo nuevo)
- Hacía siempre addLast en lugar de insertarEnAbiertos en Busqueda
for (int i = 0; i < sucesores.size(); i++) {
 abiertos.addLast(sucesores.get(i));
 insertarEnAbiertos(abiertos, sucesores.get(i));
}
- getSucesores no pasaba la heurística a los nodos
sucesores.add(new Nodo(f-1, c, actual, "N", actual.g+2, 0));
sucesores.add(new Nodo(f-1, c, actual, "N", actual.g+2, calcularH(f-1, c, mapa)))

4. Análisis de resultados

4.1 Tablas

MAPA_CALLEJON

ALGORITMO	COSTE	PASOS	NODOS EXPANDIDOS	MAX. ABIERTOS	TIEMPO (ms)
ANCHURA	8	8	19	2	49.95
PROFUNDIDAD	11	10	11	1	39.77
A* (Manhattan)	8	8	9	4	79.55
A* MEJORADO	8	8	9	4	91.12

MAPA_GRANDE

ALGORITMO	COSTE	PASOS	NODOS EXPANDIDOS	MAX. ABIERTOS	TIEMPO (ms)
ANCHURA	32	26	80	4	58.55
PROFUNDIDAD	35	28	78	5	55.26
A* (Manhattan)	32	26	58	6	104.71
A* MEJORADO	38	33	40	5	105.69

mapaTexto

ALGORITMO	COSTE	PASOS	NODOS EXPANDIDOS	MAX. ABIERTOS	TIEMPO (ms)
ANCHURA	6	6	11	1	36.23
PROFUNDIDAD	6	6	7	1	32.84
A* (Manhattan)	6	6	11	1	49.54
A* MEJORADO	6	6	11	1	47.66

4.2 Análisis de Optimalidad: ¿Encuentra el algoritmo de Anchura el camino de menor coste en este escenario?

Con costes uniformes siempre lo encuentra, pero con distintos costes en distintas direcciones no siempre lo hace. Explora por niveles sin tener en cuenta el coste de cada movimiento, tiene en cuenta el número de pasos. Por este motivo en el tema 3 se indica que es óptima en número de pasos porque con costes uniformes esto sería el camino de menor coste. Profundidad en cambio si falla porque elige el primer camino que encuentra sin considerar el coste.

Fijándonos en las tablas, en el MAPA_CALLEJON anchura encuentra un camino de coste 8 con 8 pasos, igual que A*, mientras que profundidad encuentra uno de coste 11 con 10 pasos yendo primero por la fila superior. En el MAPA_GRANDE se ve lo mismo: anchura da coste 32 igual que A*, pero profundidad da 35, lo que confirma que sin tener en cuenta los costes reales el algoritmo puede tomar el camino equivocado.

4.3 Análisis de A*: Comportamiento de la heurística con los nuevos costes

La heurística podría subestimar el coste real si hay muchos movimientos hacia las direcciones que tienen peso 2. Sin embargo, como la distancia Manhattan usa el coste 1, no sobreestima y encuentra la solución óptima. Es importante resaltar que esto hace que A* tenga que expandir más nodos porque la heurística es menos informada que antes. Para mejorarlo se podría hacer una distancia Manhattan que tuviera en cuenta los pesos hacia cada dirección, que es lo que he implementado como “BusquedaAMEjorado” como parte de lo que se pedía en el enunciado

“Implementar una heurística propia propuesta por el alumno que intente compensar los costes asimétricos”.

Sin embargo, mirando los resultados del A* Mejorado en el MAPA_GRANDE, da un coste de 38 en vez de 32. Esto significa que la heurística mejorada no es admisible en ese mapa. En los otros dos mapas si da el mejor resultado.

4.4 Tratamiento de repetidos

En el tema 3, el pseudocódigo incluye “QuitarRepetidos(sucesores, Cerrados)”, por lo que en mi código que implementado el tratamiento de repetidos de esa misma forma, en la clase Búsqueda, comprobando que el sucesor ni esté ya ni en abiertos ni en cerrados y mostrando por pantalla el repetido descartado. Esto hace que el algoritmo no entre en un bucle infinito

5. Conclusiones

La conclusión más significativa de esta práctica es que para laberintos con pesos mayores hacia arriba y hacia la izquierda la recomendación es usar A* con una heurística que tenga en cuenta los costes reales de cada dirección, no una Manhattan uniforme. Además hemos comprobado que el anchura es fiable pero expande muchos nodos y que el profundidad es el que peor funciona si tenemos costes distintos porque no los tiene en cuenta. Para comprobar todas estas conclusiones, se puede consultar el siguiente anexo (especialmente en los nodos expandidos y tiempo de ejecución)

6. Anexo: estadísticas finales con mapaTexto

```
--- ESTADISTICAS FINALES ---
Solucion encontrada usando Anchura
Camino recorrido (x,y): [[1, 1], [1, 2], [1, 3], [1, 4], [1, 5], [2, 5], [3, 5]]
Coste total del camino: 6
Nodos expandidos: 11
Tamaño maximo de ABIERTOS: 1
Tiempo de ejecucion: 36.2267 ms.
Agente decide: E
```

```
--- ESTADISTICAS FINALES ---
Solucion encontrada usando Profundidad
Camino recorrido (x,y): [[1, 1], [1, 2], [1, 3], [1, 4], [1, 5], [2, 5], [3, 5]]
Coste total del camino: 6
Nodos expandidos: 7
Tamaño maximo de ABIERTOS: 1
Tiempo de ejecucion: 32.842 ms.
Agente decide: E
```

--- ESTADISTICAS FINALES ---

Solucion encontrada usando A

Camino recorrido (x,y): [[1, 1], [1, 2], [1, 3], [1, 4], [1, 5], [2, 5], [3, 5]]

Coste total del camino: 6

Nodos expandidos: 11

Tamaño maximo de ABIERTOS: 1

Tiempo de ejecucion: 49.5422 ms.

Agente decide: E

--- ESTADISTICAS FINALES ---

Solucion encontrada usando A Mejorado

Camino recorrido (x,y): [[1, 1], [1, 2], [1, 3], [1, 4], [1, 5], [2, 5], [3, 5]]

Coste total del camino: 6

Nodos expandidos: 11

Tamaño maximo de ABIERTOS: 1

Tiempo de ejecucion: 47.6585 ms.

Agente decide: E